

```
import os
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
dataset_path = "/content/drive/MyDrive/ANNlab/Pistachio_Image_Dataset/Pistachio_Image_Dataset"
```

```
# Define image transformations
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.125])
])
```

```
class PistachioDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.images = []
        self.labels = []

        #Load images from "Kirmizi_Pistachio" folder (Class 0)
        kirmizi_folder = os.path.join(root_dir, "Kirmizi_Pistachio")
        for file in os.listdir(kirmizi_folder):
            if file.lower().endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(kirmizi_folder, file))
                self.labels.append(0) # 0 for Kirmizi_Pistachio

        # Load images from "Siirt_Pistachio" folder (Class 1)
        siirt_folder = os.path.join(root_dir, "Siirt_Pistachio")
        for file in os.listdir(siirt_folder):
            if file.lower().endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(siirt_folder, file))
                self.labels.append(1) # 1 for Siirt_Pistachio

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        # Load image and label
        img_path = self.images[idx]
        label = self.labels[idx]

        # Open image
        image = Image.open(img_path).convert("L")

        # Apply transformations
        if self.transform:
            image = self.transform(image)

        # Convert label to tensor
        label = torch.tensor(label, dtype=torch.long)

        return image, label
```

```
# Instantiate dataset
dataset = PistachioDataset(dataset_path, transform=transform)
```

```
# Create DataLoader
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)
```

```
# Test the DataLoader
for images, labels in dataloader:
    print(f"Batch size: {images.shape}, Labels: {labels}")
    break
```

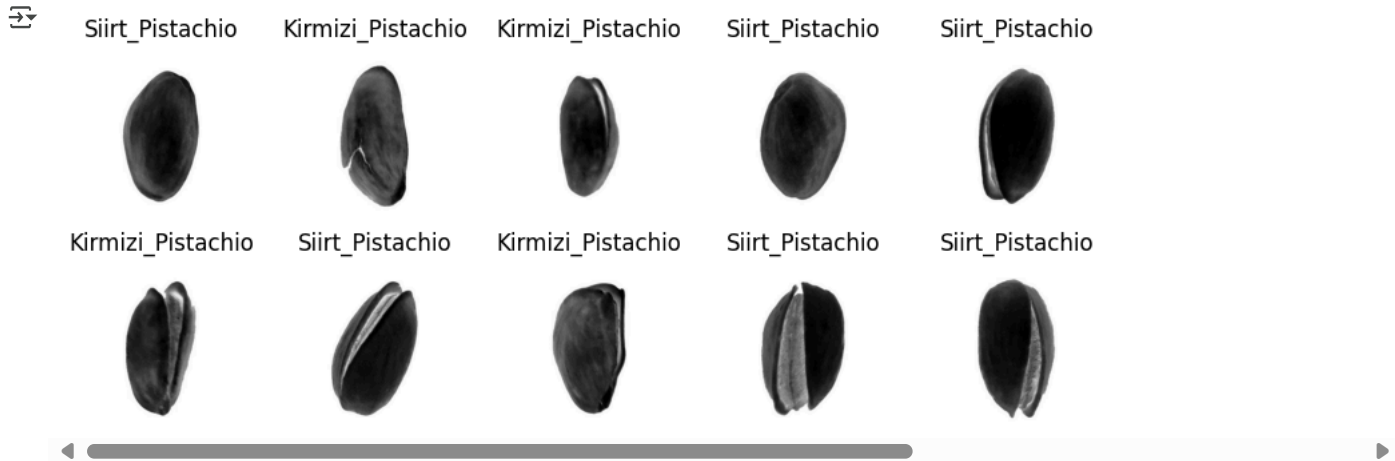
Batch size: torch.Size([64, 1, 256, 256]), Labels: tensor([0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0])

```

image, labels = next(iter(dataloader))
class_name = ['Kirmizi_Pistachio', 'Siirt_Pistachio']
plt.figure(figsize = (10,10))
for i in range(10):
    plt.subplot(5,5,i+1)
    img = image[i].permute(1,2,0)
    img = img *0.123 + 0.5
    plt.imshow(img.numpy(), cmap = 'Grays')
    plt.title(f"{class_name[labels[i].item()]}")
    plt.axis("off")

plt.show()

```



```

dataset = PistachioDataset(dataset_path, transform=transform)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)

```

```

import torch.nn as nn
class PistachioCNN(nn.Module):
    def __init__(self):
        super(PistachioCNN, self).__init__()
        self.model = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Flatten(),
            nn.Linear(64 * 32 * 32, 128),
            nn.ReLU(),
            nn.Linear(128, 2)
        )

    def forward(self, x):
        return self.model(x)

# Instantiate model
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = PistachioCNN().to(device)

# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Train model
for epoch in range(50):
    for images, labels in dataloader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()

```

```

optimizer.step()

print("Training completed.")

# =====
# Classification output visualization
# =====
class_names = ['Kirmizi_Pistachio', 'Siirt_Pistachio']

# Get a batch from dataloader
images, labels = next(iter(dataloader))
images = images.to(device)
labels = labels.to(device)

# Model prediction
outputs = model(images)
_, predicted = torch.max(outputs, 1)

# Plot first 10 predictions
plt.figure(figsize=(12, 12))
for i in range(10):
    plt.subplot(5, 5, i + 1)
    img = images[i].cpu().permute(1, 2, 0) # [C,H,W] -> [H,W,C]
    img = img * 0.125 + 0.5 # Denormalize
    img = img.squeeze() # Remove channel dimension
    plt.imshow(img.numpy(), cmap='gray')
    plt.title(f"Actual: {class_names[labels[i]]}\nPred: {class_names[predicted[i]]}")
    plt.axis("off")

plt.tight_layout()
plt.show()

```

...

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

...

Start coding or generate with AI.